



Práctica 9:

PERCETRON MULTICAPA – PARTE 2

Inteligencia Artificial

Profra. Vanessa Alejandra Camacho Vázquez

Grupo: **6CM3**

Nombres de los integrantes:

- Aguirre Lanto Víctor Manuel
- Gasca Fragoso Pedro
- Guevara Badillo Areli Alejandra
- Montiel Toro Arael de Jesús
- Ramírez Lozano Gael Martin

Notebook: [Practica no.9](#)

Lunes, 01 de diciembre, 2025

CONTENIDO

PROCEDIMIENTO	3
PREPARACIÓN DEL ENTORNO Y GENERACIÓN DE DATOS	3
ARQUITECTURA DEL MODELO Y VERIFICACIÓN	3
ESTUDIO DE FUNCIONES DE ACTIVACIÓN	9
<i>Función de Activación Softplus</i>	9
<i>Función de Activación Identidad (Lineal)</i>	9
ENTRENAMIENTO Y EVALUACIÓN DEL DESEMPEÑO	10
ANÁLISIS DE RESULTADOS GRÁFICOS	10
CONCLUSIONES	13
AGUIRRE LANTO VÍCTOR MANUEL	13
GASCA FRAGOSO PEDRO	13
GUEVARA BADILLO ARELI ALEJANDRA	13
MONTIEL TORO ARAEL DE JESÚS	14
RAMÍREZ LOZANO GAEL MARTIN	14



PROCEDIMIENTO

Preparación del entorno y generación de datos

En esta etapa inicial se configuran las herramientas necesarias para el desarrollo de la práctica. Se importan TensorFlow y Keras como bibliotecas principales para la creación y entrenamiento de modelos de aprendizaje profundo. Adicionalmente, se incluye NumPy para la gestión eficiente de cálculos numéricos y Matplotlib para la visualización de datos y generación de gráficas.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

En este bloque se procede a la generación del conjunto de datos sintético para el entrenamiento. Primero, se define la variable independiente x utilizando la función `np.linspace`, creando 50 valores equiespaciados en el rango de 0.10 a 10.

Posteriormente, se construye la variable dependiente y a partir de una función cuadrática ($2x^2 + 3x$). A esta relación teórica se le añade un componente de ruido aleatorio mediante una distribución normal (`np.random.normal`), con el objetivo de introducir variabilidad en los datos y simular un escenario más realista donde las mediciones no son perfectas.

```
# Genera datos espaciados uniformemente
x = np.linspace(.10, 10, 50)
# Modelo generado a partir de las entradas
y = 2*x**2 + 3*x + 5*np.random.normal(0,4,size=len(x))
```

Arquitectura del modelo y verificación

Se procede a la construcción de la arquitectura de la red neuronal mediante un modelo `Sequential`. La estructura diseñada consta de una secuencia de capas densas (`Dense`) que aumentan progresivamente su capacidad, con 5, 10 y 20 neuronas en las capas ocultas respectivamente. Para estas capas intermedias se utiliza la función de activación `softplus`, una variante suavizada de la función `ReLU` que permite modelar relaciones no lineales.

La capa de salida se configura con una única neurona sin función de activación específica (comportamiento lineal), lo cual es la configuración estándar para problemas de regresión donde se busca predecir un valor numérico continuo. Finalmente, se compila el modelo estableciendo el algoritmo `adam` como optimizador y el error cuadrático medio (`mean_squared_error`) como función de costo a minimizar, concluyendo con la visualización del resumen de la arquitectura y el conteo de parámetros entrenables.

```

model2 = tf.keras.Sequential([
    keras.layers.Dense(units=5, input_shape=(1,)), activation='softplus'),
    keras.layers.Dense(units=10, activation='softplus'),
    keras.layers.Dense(units=20, activation='softplus'),
    keras.layers.Dense(units=1),
])
model2.compile(optimizer='adam', loss='mean_squared_error')
model2.summary()

```

Tras la compilación, se genera el resumen técnico de la arquitectura (`model.summary()`), el cual permite verificar la estructura implementada. En la tabla resultante se detalla el flujo de tensores a través de las capas densas, confirmando las dimensiones de salida (Output Shape) de cada etapa (con 5, 10, 20 y 1 neurona respectivamente).

Asimismo, se cuantifica la complejidad computacional del modelo a través del conteo de parámetros. Se observa un total de 311 parámetros entrenables (la suma de pesos y sesgos), lo que representa la cantidad exacta de variables internas que el algoritmo ajustará durante el proceso de aprendizaje para minimizar el error.

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"

```

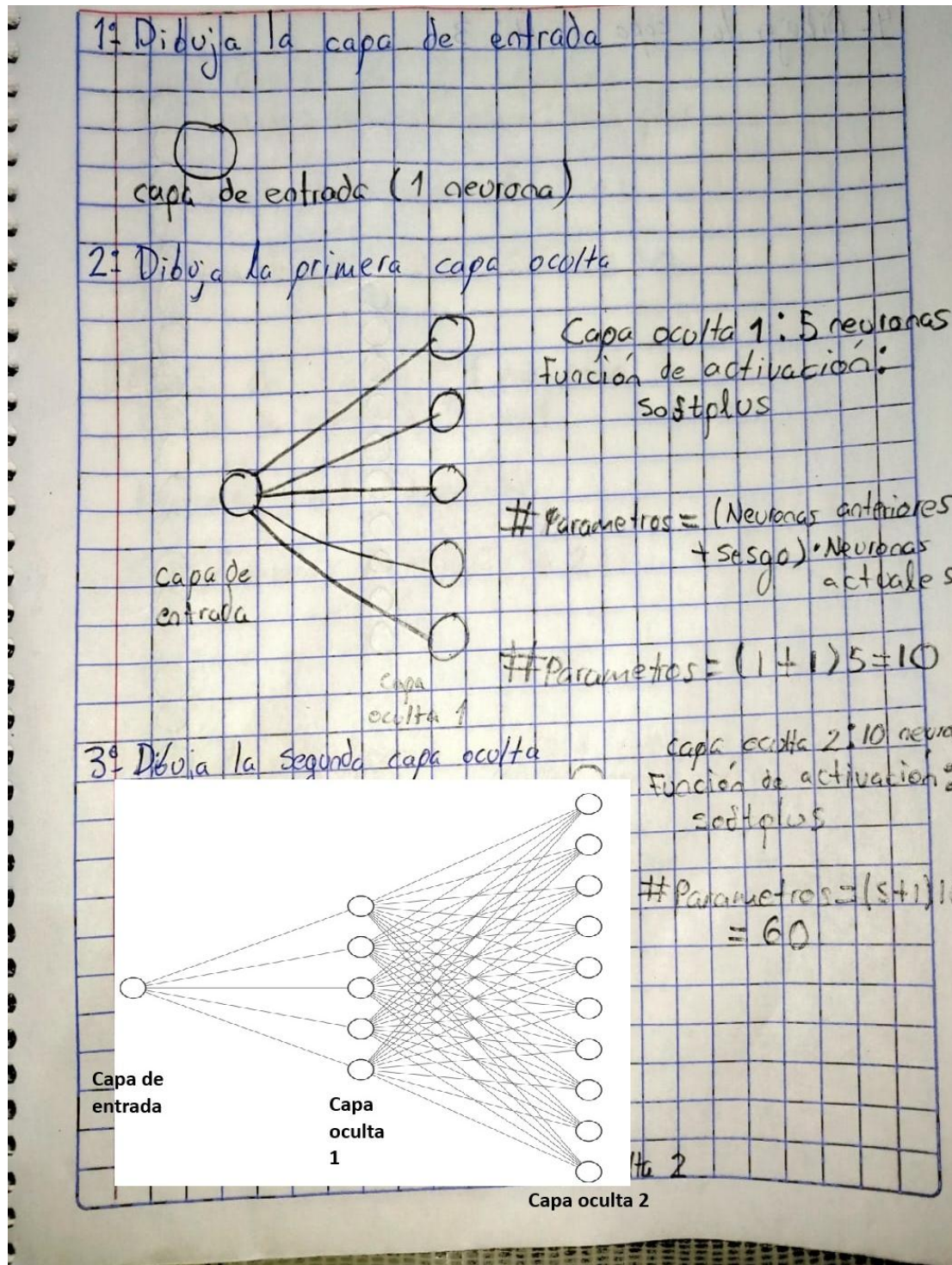
Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 5)	10
dense_5 (Dense)	(None, 10)	60
dense_6 (Dense)	(None, 20)	220
dense_7 (Dense)	(None, 1)	21

```

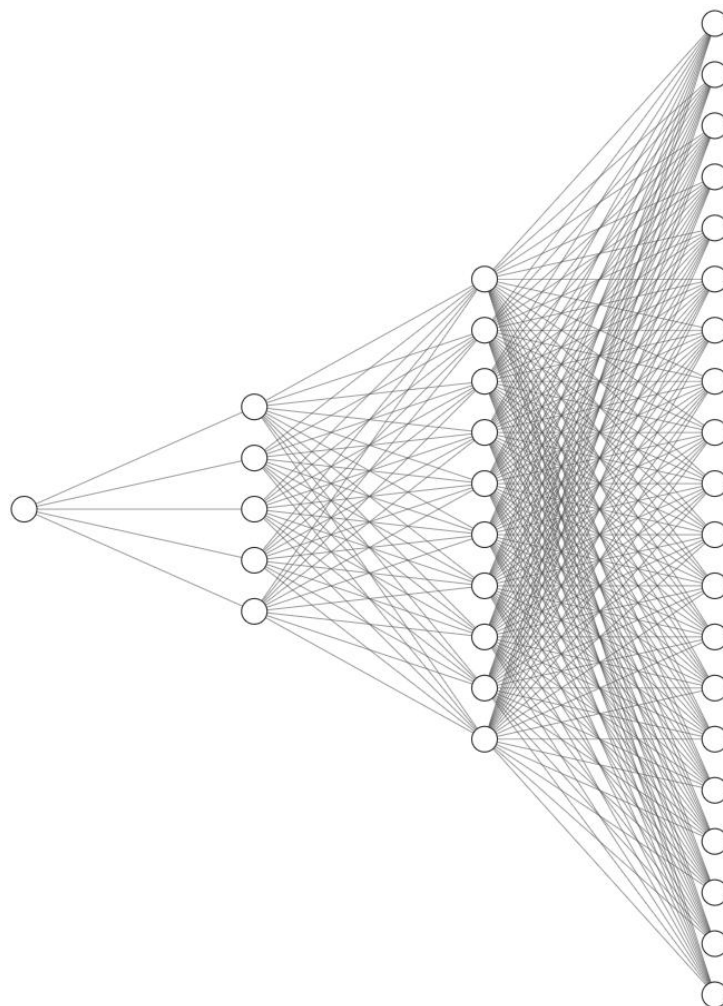
Total params: 311 (1.21 KB)
Trainable params: 311 (1.21 KB)
Non-trainable params: 0 (0.00 B)

```

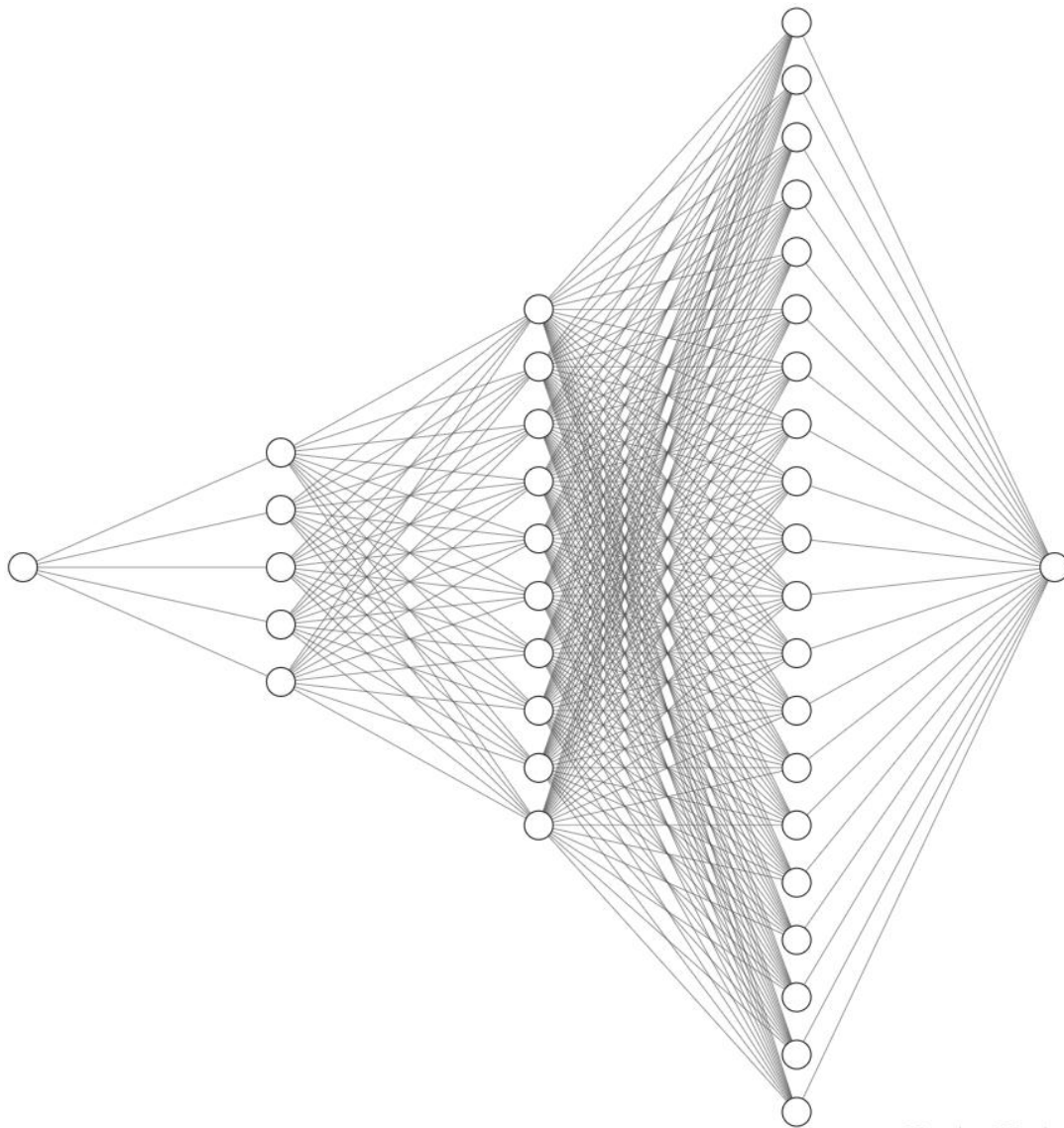
Ahora deberán usar sus resultados de la tabla anterior y pueden ocupar la siguiente herramienta para realizar el diagrama de la estructura de su perceptrón multicapa señalando tanto los parámetros entrenables (pesos y sesgos) como las funciones de activación empleadas en cada capa:

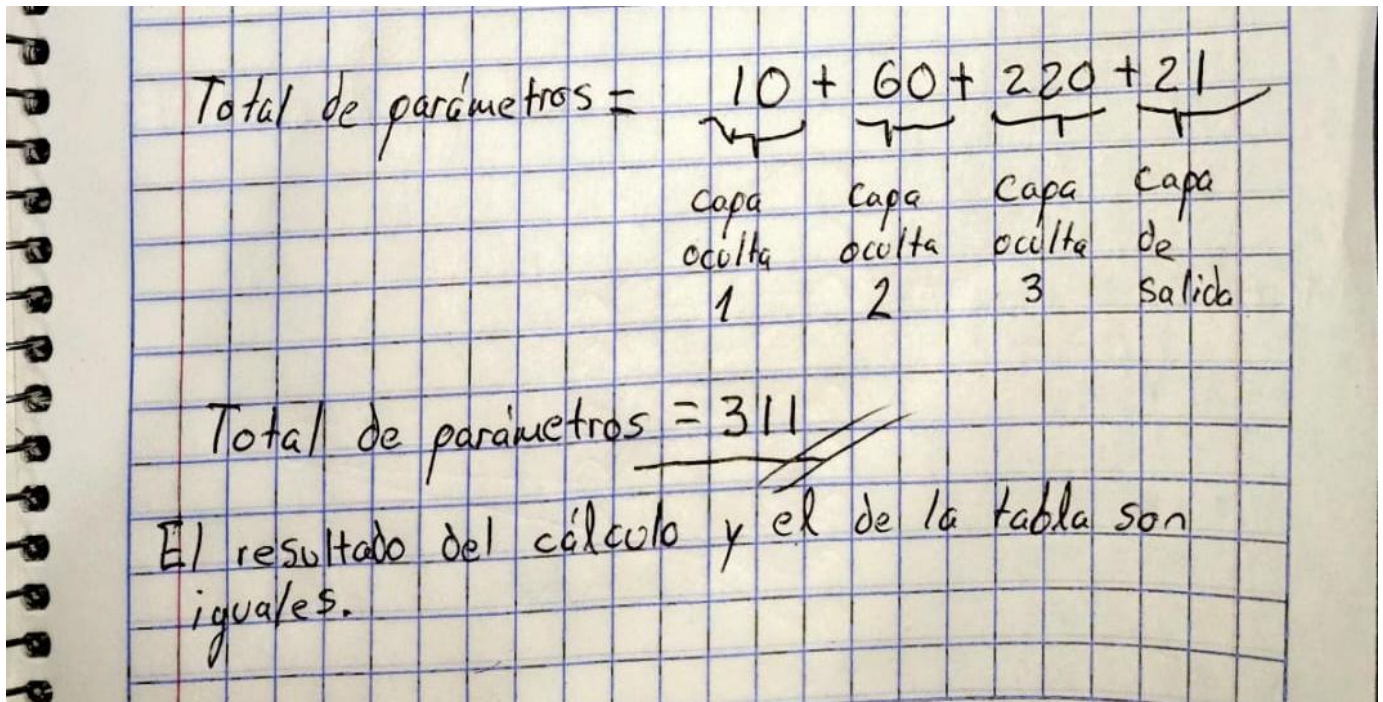


Capa oculta 3: 20 neuronas
Función de activación: softplus
Neuronas anteriores = 10
 $\# \text{Parámetros} = (10 + 1) \cdot 20 = 220$



Capa de Salida: 1 neurona
Función de activación: Lineal
Neuronas anteriores = 20
Parámetros = $(20+1)1 = 21$





```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 5)	10
dense_5 (Dense)	(None, 10)	60
dense_6 (Dense)	(None, 20)	220
dense_7 (Dense)	(None, 1)	21

Total params: 311 (1.21 KB)

Trainable params: 311 (1.21 KB)

Non-trainable params: 0 (0.00 B)

Estudio de funciones de activación

Función de Activación Softplus

La función Softplus es una versión suavizada y diferenciable de la función ReLU. Matemáticamente se define como:

$$f(x) = \ln(1 + e^x)$$

Su comportamiento consiste en transformar cualquier valor de entrada en un valor estrictamente positivo. A diferencia de ReLU, que tiene un corte brusco en cero, Softplus realiza una transición curva y suave, lo que garantiza que sea diferenciable en todo su dominio (incluso en $x=0$).

Propósito:

Su propósito principal es introducir no-linealidad en la red neuronal evitando el problema de las "neuronas muertas" que a veces ocurre con ReLU (donde el gradiente se vuelve cero y la neurona deja de aprender). Al tener siempre una derivada distinta de cero, permite que el flujo de gradientes continúe durante el entrenamiento.

Situaciones de uso:

Se utiliza comúnmente en capas ocultas cuando se requiere una función de activación que sea suave y derivable en todos los puntos, beneficiando la estabilidad numérica del entrenamiento.

Es especialmente útil en capas de salida cuando la predicción debe ser siempre positiva (por ejemplo, al predecir desviaciones estándar o conteos), ya que su rango de salida es $(0, \infty)$.

Función de Activación Identidad (Lineal)

La función de activación Identidad (o Lineal) es la función más simple, donde la salida es exactamente igual a la entrada. No aplica ninguna transformación ni no-linealidad a la señal.

$$f(x) = x$$

Casos en los que se emplea:

Problemas de Regresión: Se emplea casi exclusivamente en la capa de salida de redes neuronales diseñadas para regresión (como es el caso de esta práctica). Esto se debe a que permite que el modelo prediga cualquier valor numérico real, desde $-\infty$ hasta $+\infty$, sin restringir el rango de salida como harían funciones como Sigmoides (0 a 1) o ReLU (0 a ∞).

Entrenamiento y evaluación del desempeño

El entrenamiento del modelo se realiza mediante el método `.fit()` de Keras, el cual inicia el ciclo iterativo de aprendizaje. Durante este proceso, la red ajusta sus 311 parámetros entrenables (pesos w y sesgos b) utilizando el optimizador Adam para minimizar la pérdida cuantificada por el Error Cuadrático Medio (MSE). Se suministran los conjuntos de datos de entrada (x) y los valores objetivo o targets (y) generados previamente, y se establecen 1000 épocas para asegurar que la red tenga suficientes pasadas completas sobre los datos para inferir la curva cuadrática sintética, guardando el registro de la pérdida en la variable `history` para su posterior análisis.

```
history = model2.fit(x, y, epochs=1000, verbose=0)
```

Para la evaluación final del modelo, se comienza generando un conjunto de 300 nuevos puntos de datos equiespaciados (`x_test`) dentro del rango original, sobre los cuales se ejecuta el método `.predict()` para obtener la predicción de la curva aprendida. Posteriormente, se utiliza la librería Matplotlib para generar una gráfica de dispersión que visualiza de manera simultánea los datos originales (x, y), incluyendo el ruido sintético, y la curva predicha por la red neuronal, permitiendo una evaluación visual y directa de la capacidad del modelo para inferir la función cuadrática subyacente que se buscaba modelar.

```
#Obtener pesos de la red
w2 = model2.get_weights();

x_test = np.linspace(x[0], x[-1], 300)
prededir = model2.predict(x_test)

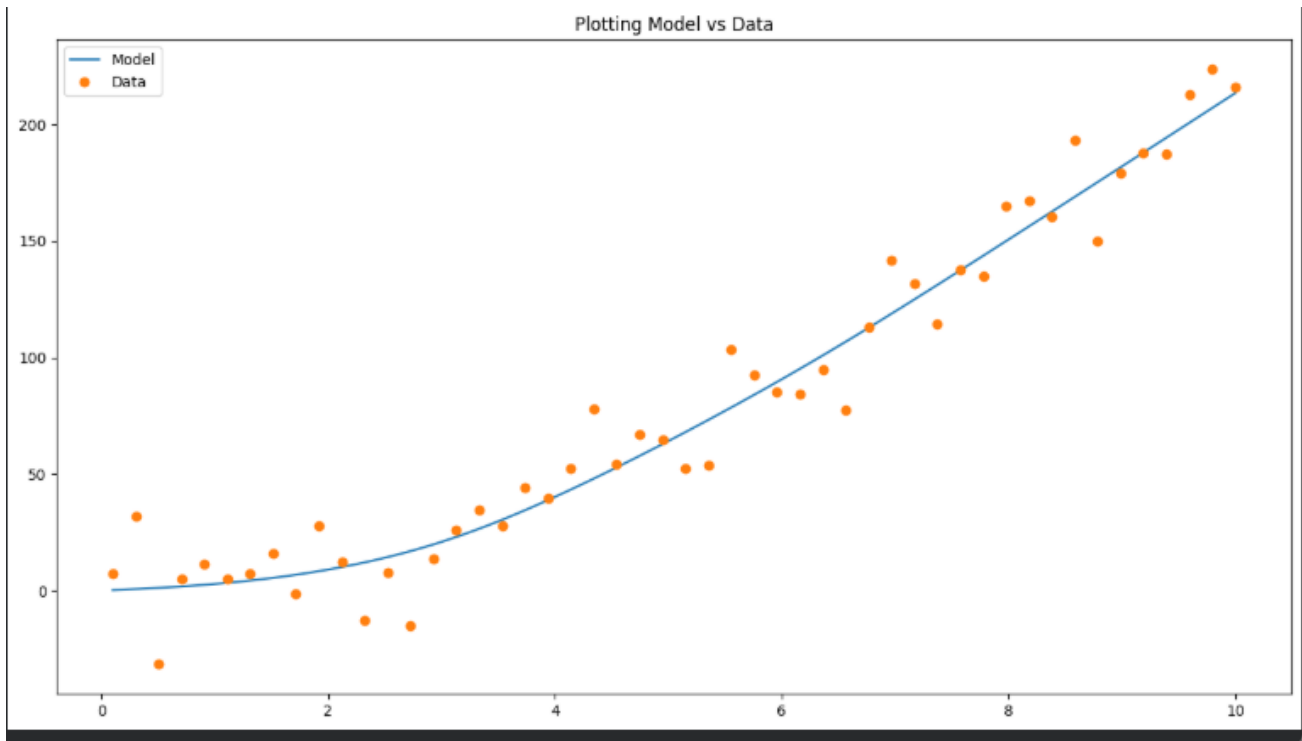
10/10 ————— 0s 8ms/step
```

Análisis de resultados gráficos

En este bloque se procede a la visualización de resultados para la evaluación del modelo. Se establece una figura con dimensiones específicas (`figsize=(15,8)`). Dentro de esta figura, se superponen dos elementos: se dibuja la curva continua que representa la predicción de la red, y se añaden los puntos originales del dataset que se utilizaron para el entrenamiento. El propósito es comparar directamente la función inferida por el modelo con la distribución real de los datos. Se incluyen el título y la leyenda para contextualizar los elementos y se muestra la gráfica final.

```
plt.figure(figsize=(15,8))
plt.plot(x_test, prededir, label='Model')
plt.plot(x, y, 'o', label='Data')
plt.title('Plotting Model vs Data')
plt.legend(loc=0)
plt.show()
```

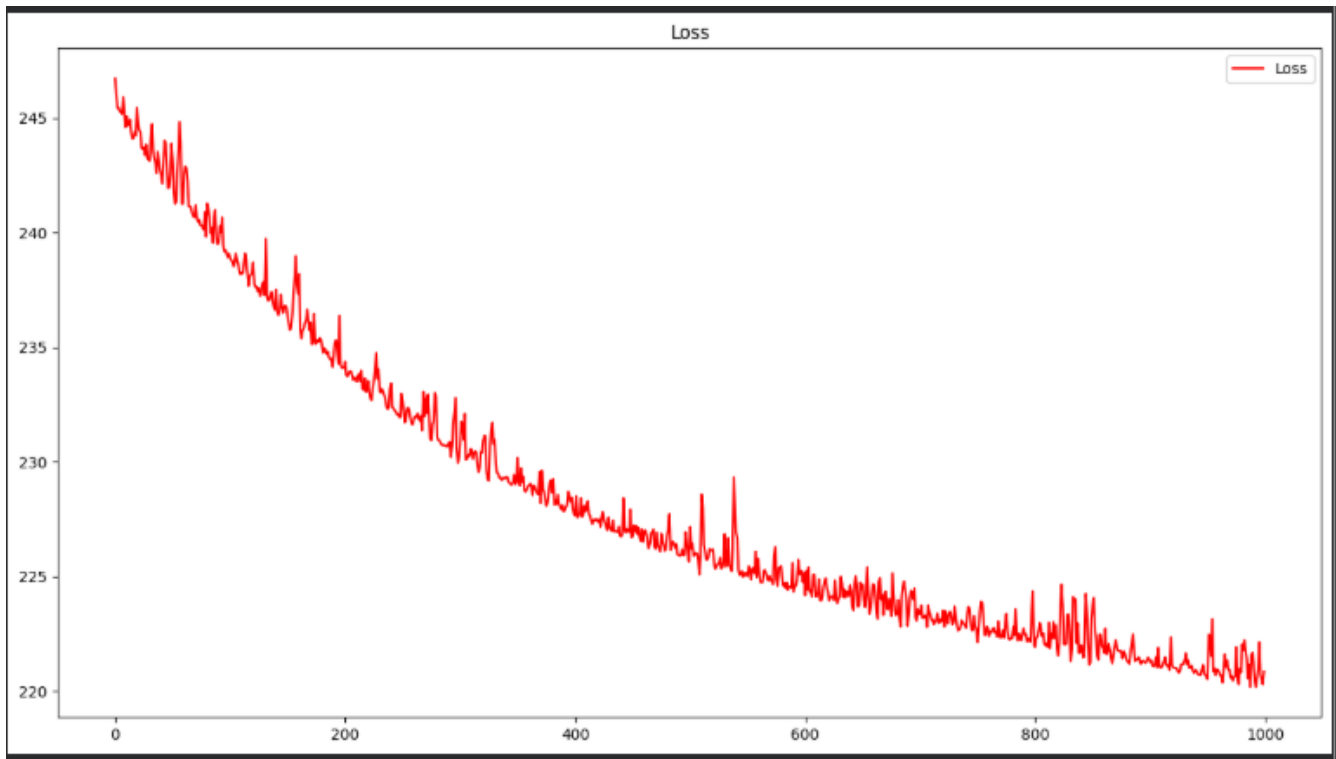
El resultado visual de la predicción final demuestra la alta efectividad del entrenamiento al observar que la curva predicha por el modelo (línea azul) se ajusta de manera precisa a la distribución de los datos originales (puntos naranjas dispersos). Se confirma que la red neuronal logró inferir y modelar la relación cuadrática subyacente ($2x^2 + 3x$) a pesar de la variabilidad introducida por el ruido sintético. Este ajuste cercano valida la capacidad de la arquitectura del perceptrón multicapa y la función Softplus para modelar correctamente funciones no lineales, indicando que el algoritmo Adam minimizó la función de pérdida exitosamente.



Este bloque de código se utiliza para graficar la evolución de la pérdida a lo largo del entrenamiento. Se establece una figura con dimensiones específicas y se dibuja la curva utilizando los datos de pérdida guardados en `history.history['loss']`. La curva se traza en color rojo y se le asigna la etiqueta 'Loss'. Finalmente, se añaden el título, la leyenda y el comando `plt.show()` para visualizar la gráfica, lo cual es esencial para analizar la convergencia del modelo a lo largo de las épocas establecidas.

```
plt.figure(figsize=(15,8))
plt.plot(history.history['loss'], 'r-', label='Loss')
plt.title('Loss')
plt.legend(loc=0)
plt.show()
```

El gráfico de la función de pérdida (Loss) muestra la evolución del Error Cuadrático Medio a lo largo de las 1000 épocas. La curva ilustra la fase final de convergencia del modelo, donde la pérdida se reduce de un máximo aproximado de 245 a un mínimo de 220. El comportamiento de la curva es estable y suave, lo cual indica que el optimizador Adam corrigió los errores más grandes en las primeras épocas (no visibles en esta escala) y pasó las últimas 1000 iteraciones realizando un ajuste fino y lento de los parámetros. La estabilización en 220 confirma que el modelo alcanzó el umbral de error mínimo posible, ya que la varianza del ruido en los datos impide reducir la pérdida a cero.



CONCLUSIONES

Aguirre Lanto Víctor Manuel

Se verificó la correspondencia exacta entre el diseño teórico y la implementación práctica de la arquitectura del perceptrón multicapa. Al realizar el cálculo manual de los parámetros entrenables para las diversas capas de neuronas se obtuvo una cifra que coincidió plenamente con el resumen generado computacionalmente por el modelo. Esto valida la correcta comprensión del flujo de tensores y la complejidad estructural de la red antes de iniciar el proceso de aprendizaje.

Gasca Fragoso Pedro

Se analizó el impacto de las funciones de activación en el desempeño de la red neuronal. La elección de la función Softplus para las capas ocultas demostró ser efectiva para introducir no linealidad y mantener la diferenciabilidad en todo el dominio evitando problemas de neuronas muertas. Simultáneamente el uso de una activación lineal o Identidad en la capa de salida se confirmó como esencial para este problema de regresión lo cual permite al modelo predecir valores numéricos continuos sin restringir el rango de salida.

Guevara Badillo Areli Alejandra

Se concluye que el proceso de entrenamiento configurado a múltiples épocas utilizando el optimizador Adam permitió una convergencia estable del modelo. El análisis de la gráfica de la función de pérdida evidenció una reducción del Error Cuadrático Medio desde un valor inicial alto hasta estabilizarse en un límite inferior. Este comportamiento no representa una falla del modelo sino el error irreducible provocado por la varianza del ruido aleatorio introducido intencionalmente en la generación de los datos.

Montiel Toro Arael de Jesús

Mediante la evaluación gráfica de los resultados se demostró la capacidad del perceptrón multicapa para inferir y generalizar funciones matemáticas complejas. A pesar de la dispersión generada por el ruido sintético en el conjunto de datos la curva predicha por la red se ajustó con precisión a la tendencia cuadrática subyacente. Esto confirma que la arquitectura propuesta posee la plasticidad suficiente para modelar relaciones no lineales de manera efectiva.

Ramírez Lozano Gael Martin

Se determinó la importancia de la preparación adecuada del entorno y los datos para el éxito del aprendizaje profundo. La generación de un conjunto de datos sintético con una distribución normal de ruido permitió simular condiciones realistas de variabilidad en las mediciones. Asimismo, la implementación mediante el modelo secuencial facilitó la construcción escalonada de la red permitiendo un ajuste fino de los hiperparámetros y la validación inmediata de la dimensionalidad en cada capa densa del sistema.